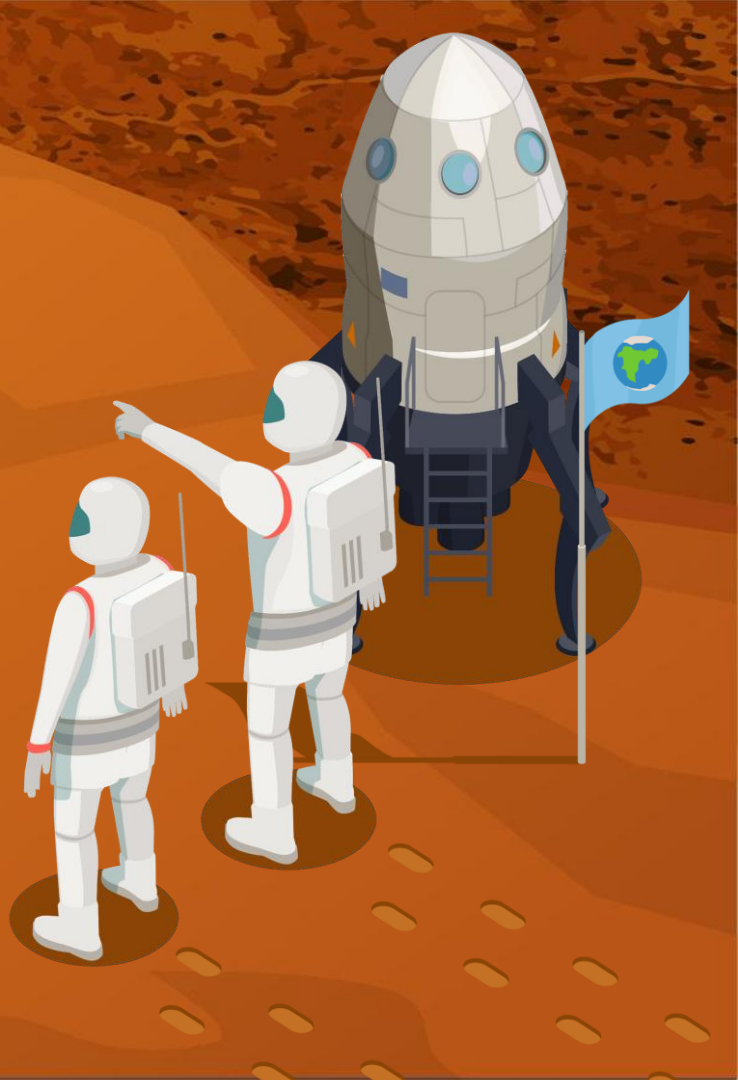
PLAN(ET) MARS

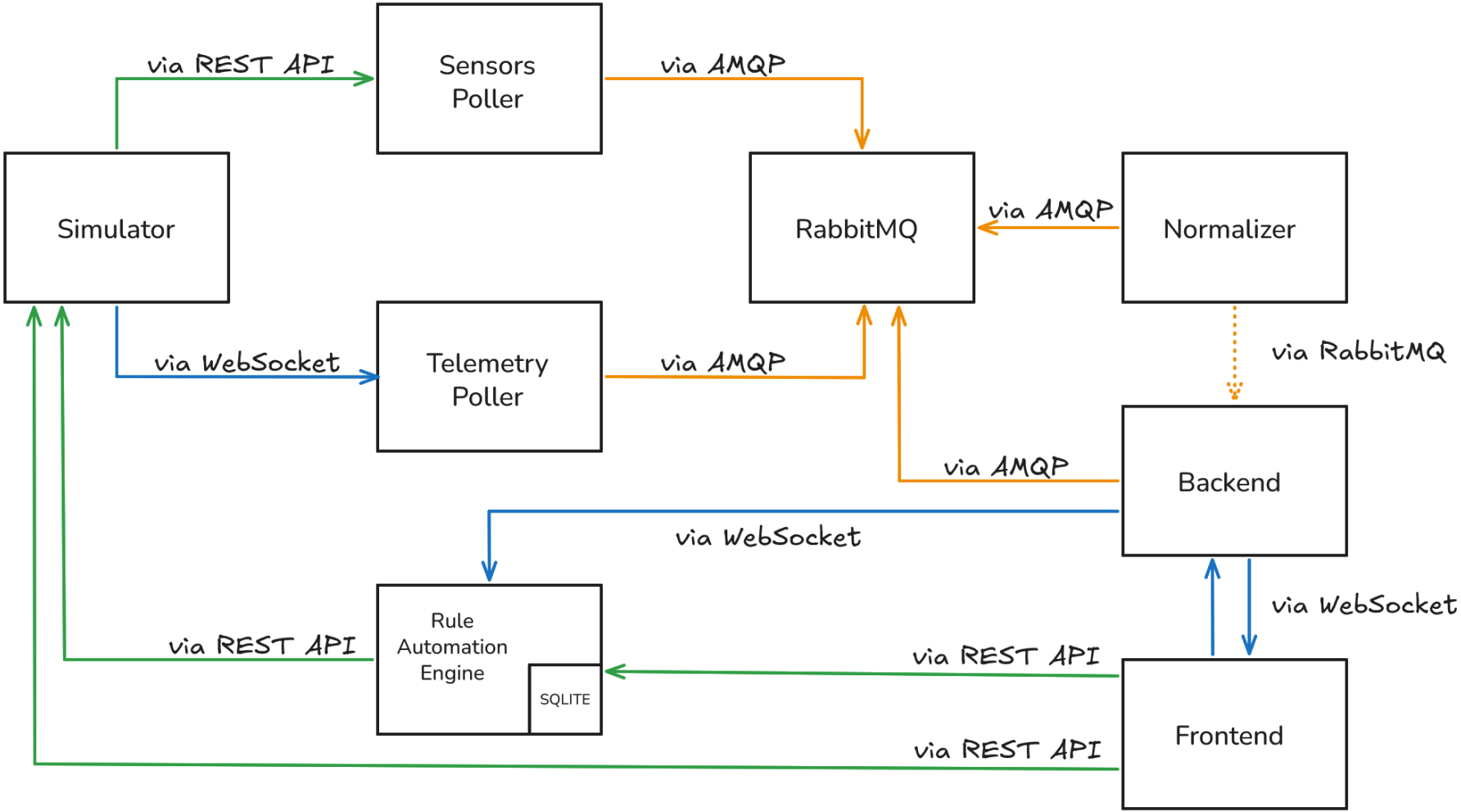
Monitoring System

Please don’t die



01

Architecture Design


via REST API
Sensors Poller
via AMQP
Simulator
RabbitMQ
via AMQP
Normalizer
via WebSocket
Telemetry Poller
via AMQP
via RabbitMQ
via AMQP
Backend
via WebSocket
via WebSocket
Rule Automation Engine
SQLITE
via REST API
via REST API
Frontend
via REST API

RabbitMQ

To fulfill the requirements of our application we employed a **publish/subscribe** mechanism. Namely we created two **exchanges**:

- **Unnormalized**: all the queues within this exchange are filled with data pushed by the injectors to the normalizer
- **Normalized**: this exchange is where the result of normalization is stored, to be used later down in the pipeline

Injectors

There are two types of injectors:

- **Sensor poller**: via **REST API** acquires data from sensors
- **Telemetry poller**: via **WebSocket** receives data from telemetry

Both of them push the data to the **unnormalized** exchange. Furthermore data schema validation is ensured with pydantic *BaseModel*.

Normalizer

This microservice is responsible for unifying the **etherogeneous** data into a **unified internal schema**.

An attempt was made to use two different schema, to distinguish between sensors and telemetry, to reduce the overhead of representing both at once, given that telemetry is far less frequent than sensor data.
Ultimately, we managed to represent both streams in a unified manner.

Backend and Frontend

For these two microservices, communication-wise, we opted for a **WebSocket** paradigm, because:

- The header size is way smaller
- The connection is persistent and full-duplex

These properties fit our **low latency** requirements.

Rule automation engine

Exactly as the frontend, it gathers the normalized data via **WebSocket** which is then used to compare against the **rules** stored in the database, taking action upon verifying the conditions, by performing a POST request to the simulator.

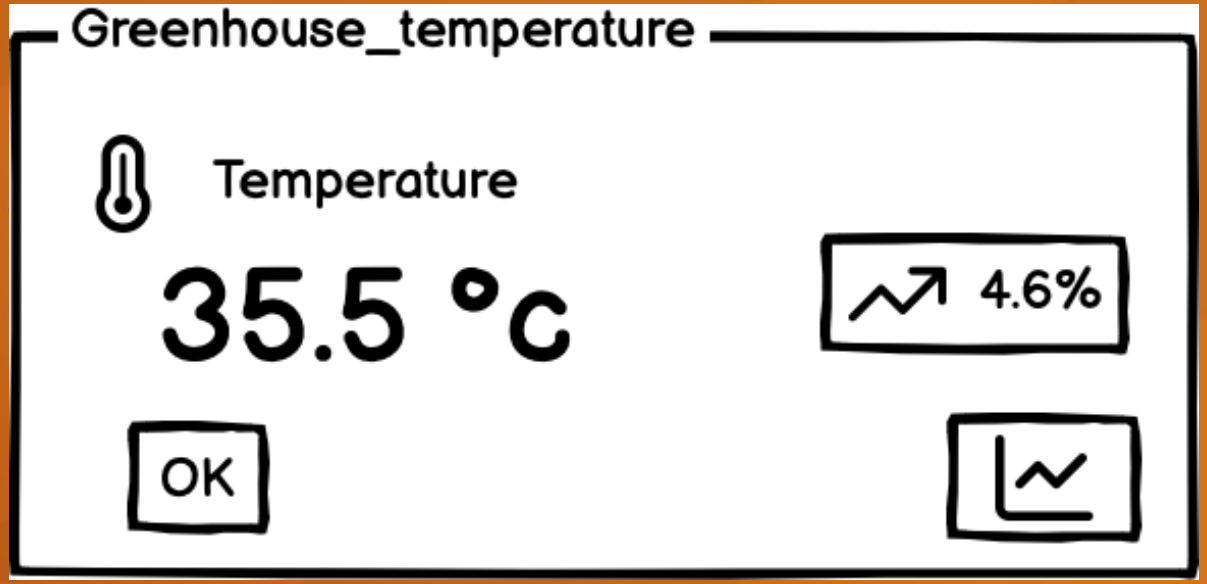
As it concerns the database, we chose **SQLite** since the application requires just one table whose cardinality will not exceed thousands of rows. In addition, we do not perform lots of writes, especially not in a concurrent manner, since we are alone on Mars.
Given these properties SQLite seemed a perfect fit for our system.

2

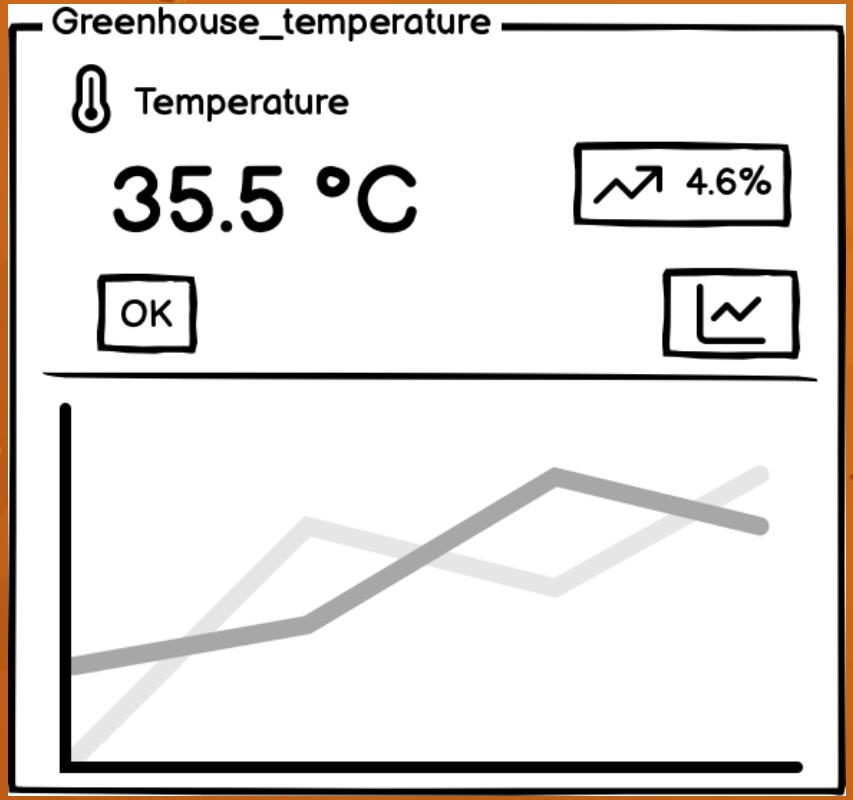
USER STORIES

Latest Data

As a user, I want to visualize the latest data reported by each sensor.



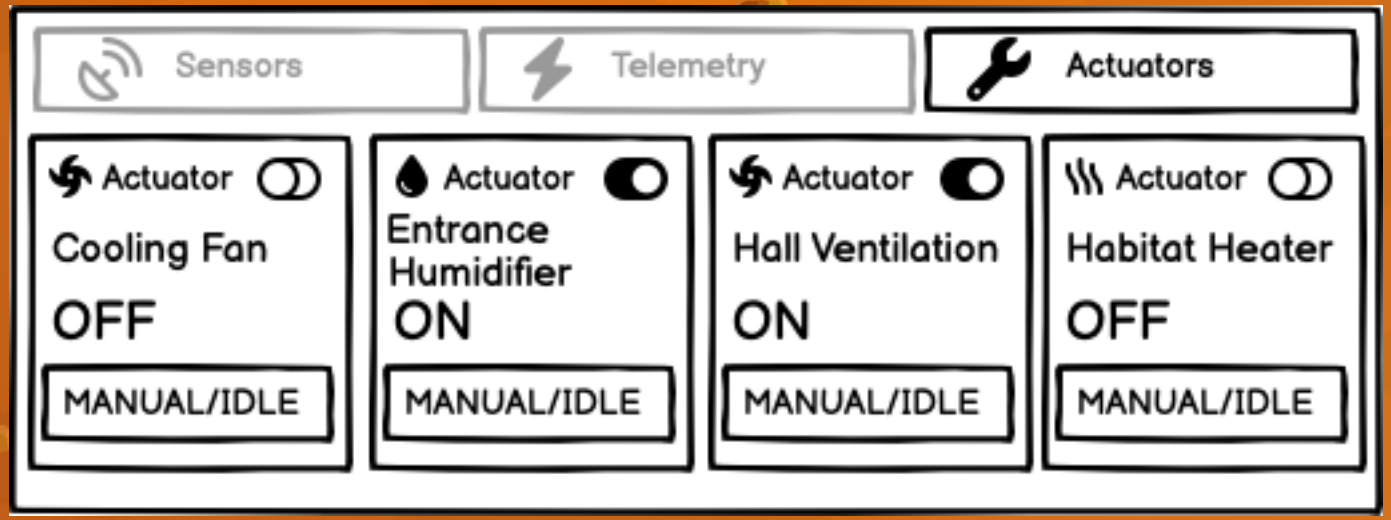
Historical values



As a user, I want to visualize the historical values for each sensor, so that I can see the evolution of the situation over time.

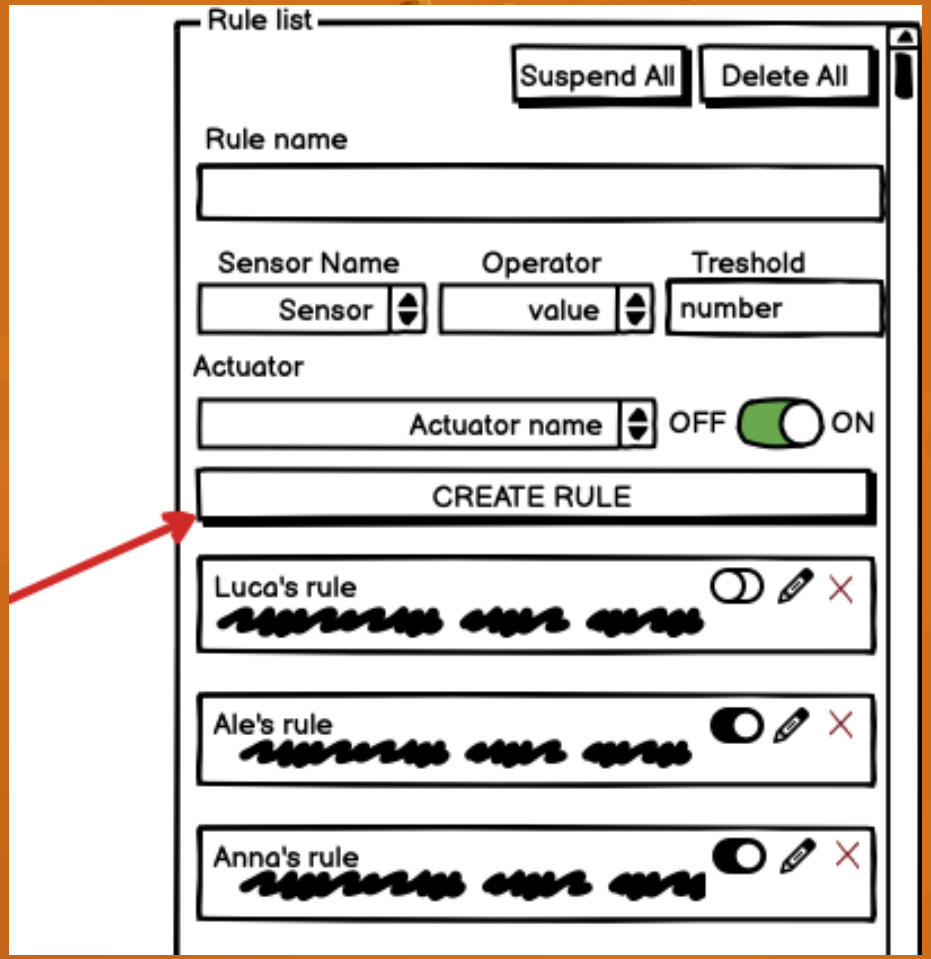
Manual actuators

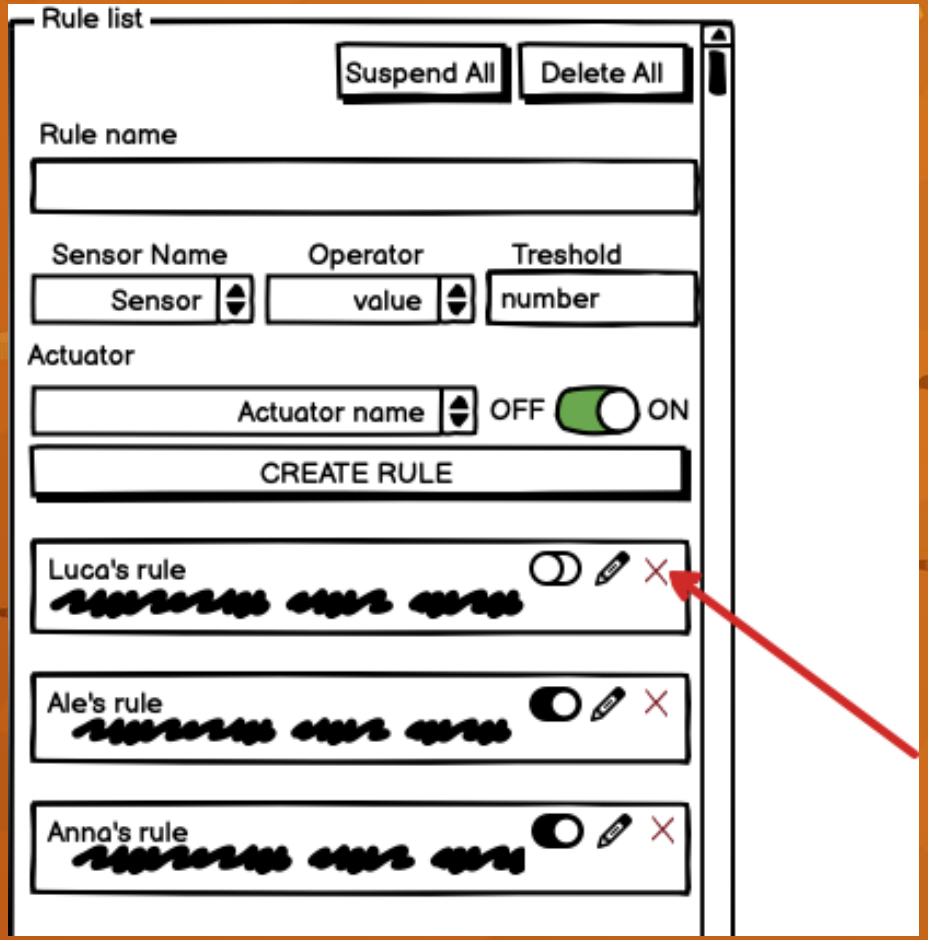
As a user, I want to be able to manually trigger actuators, so that I can take action in emergency situations.



Create rules

As a user, I want to specify rules based on values reported by the sensor and take action through the actuators, so that i can automate my work.



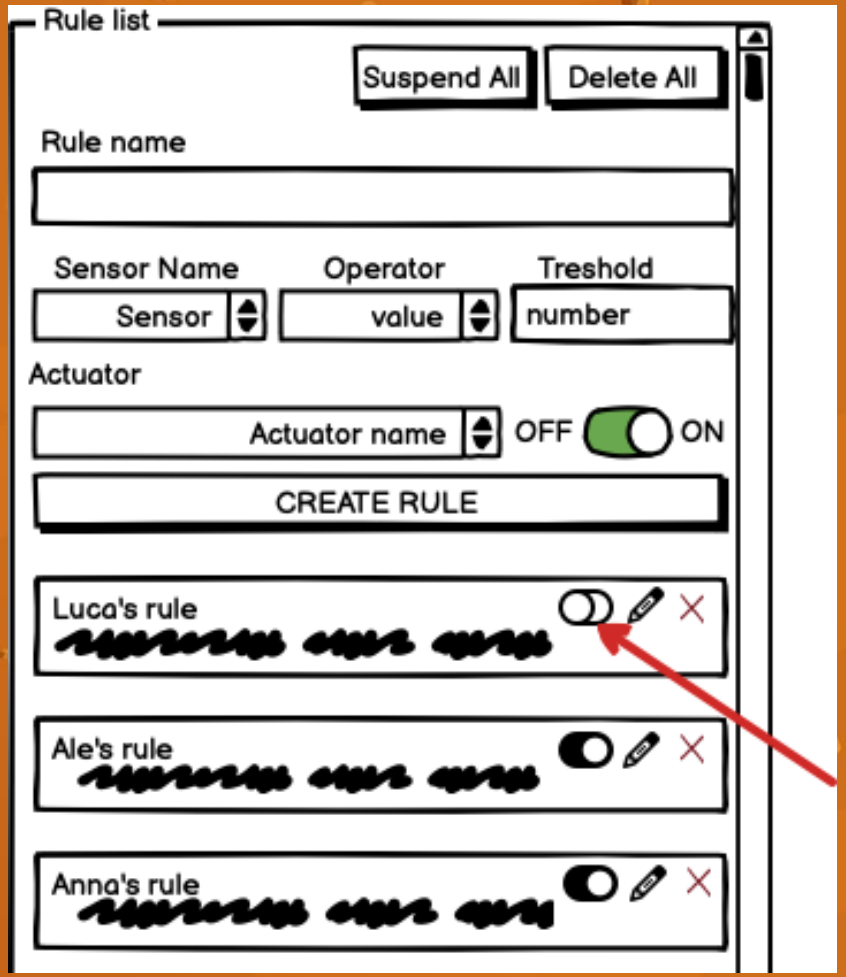


Delete rules

As a user, I want to delete previously specified rules.

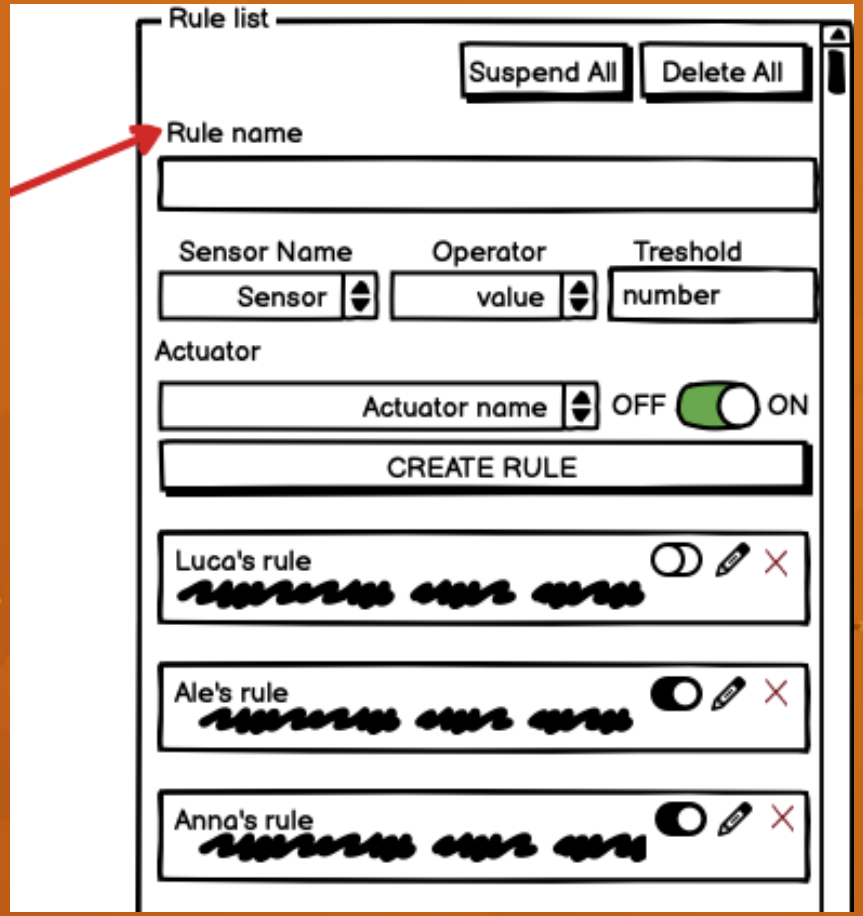
Toggle rules

As a user, I want to be able to temporarily toggle a rule, so that I can perform maintenance.



Rule activation

As a user, I want the system to identify which rule activated an actuator, so that everything is transparent.

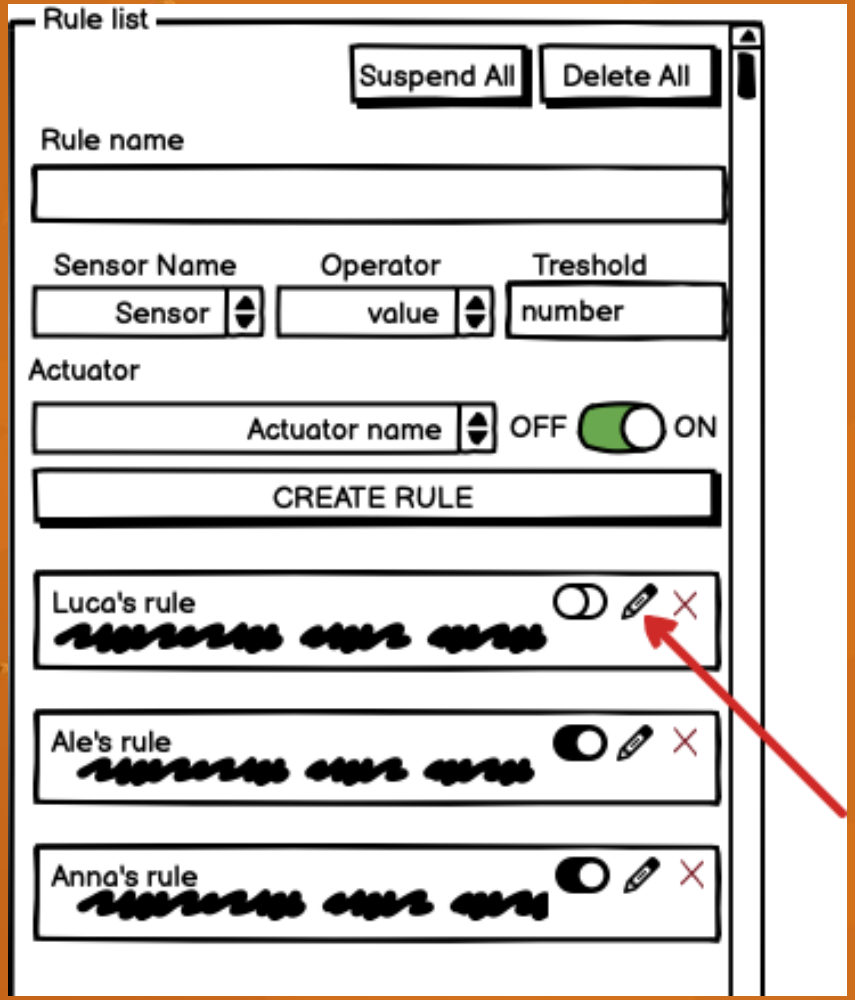


Naming rules

As a user, I want to give names to rules, so that I can easily identify rules.

Rule editing

As a user, I want to be able to edit a rule.

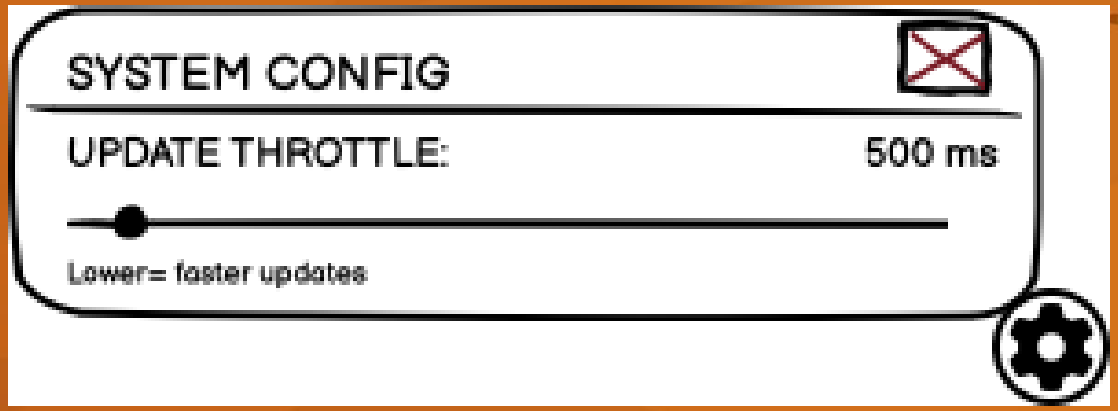


Overall state

As a user, I want to see at a glance the overall health of the system.

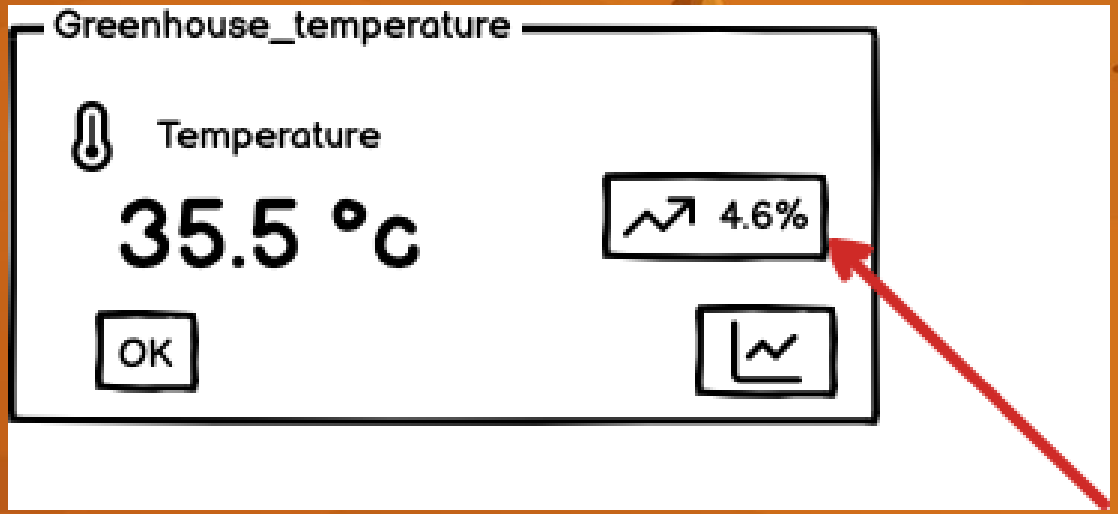
Polling rate

As a user, I want to be able to control the polling rate of the data from the sensors, so that I can reduce the power consumption of the system.



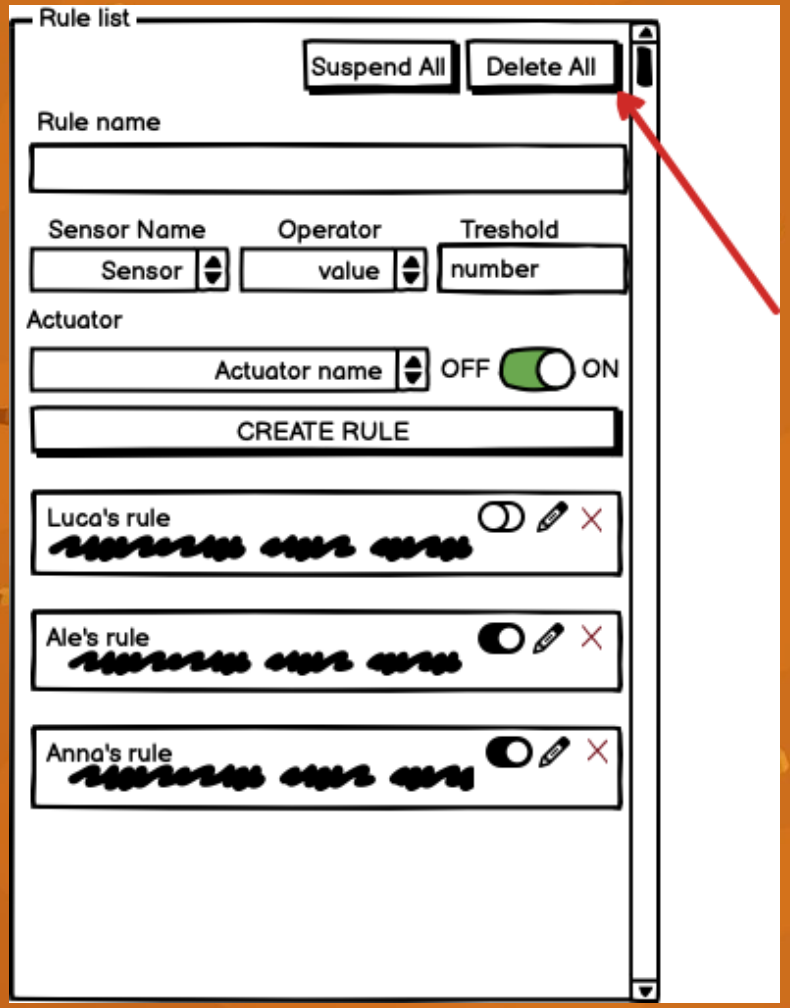
Trend

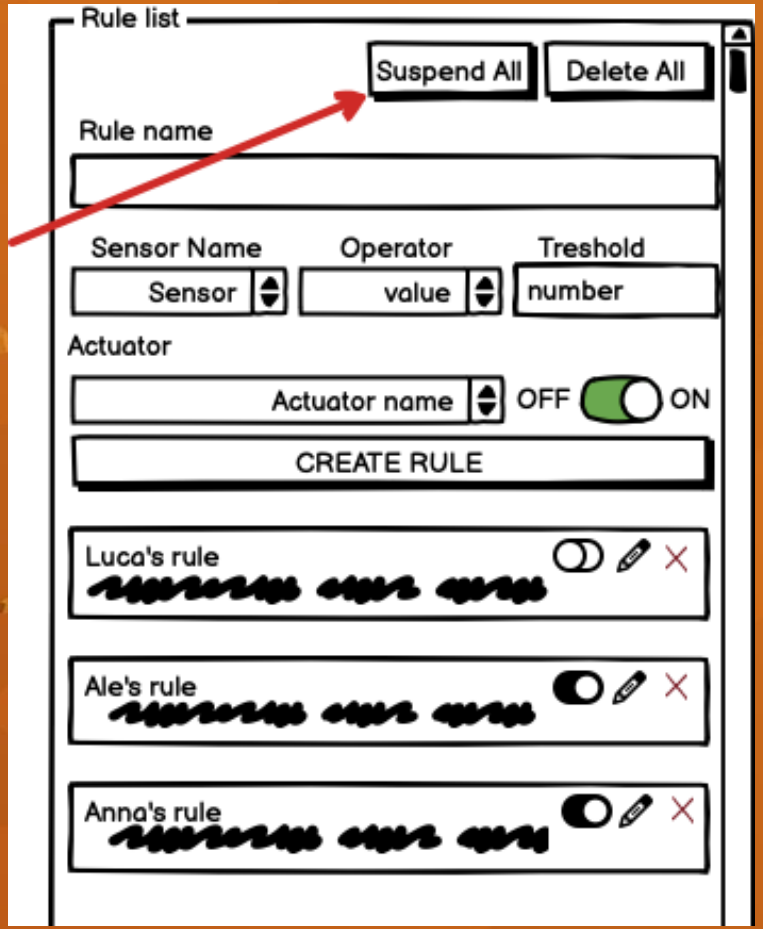
As a user, I want to be able to quickly see the trend of sensed data, so that I can quickly monitor the situation.



Delete all rules

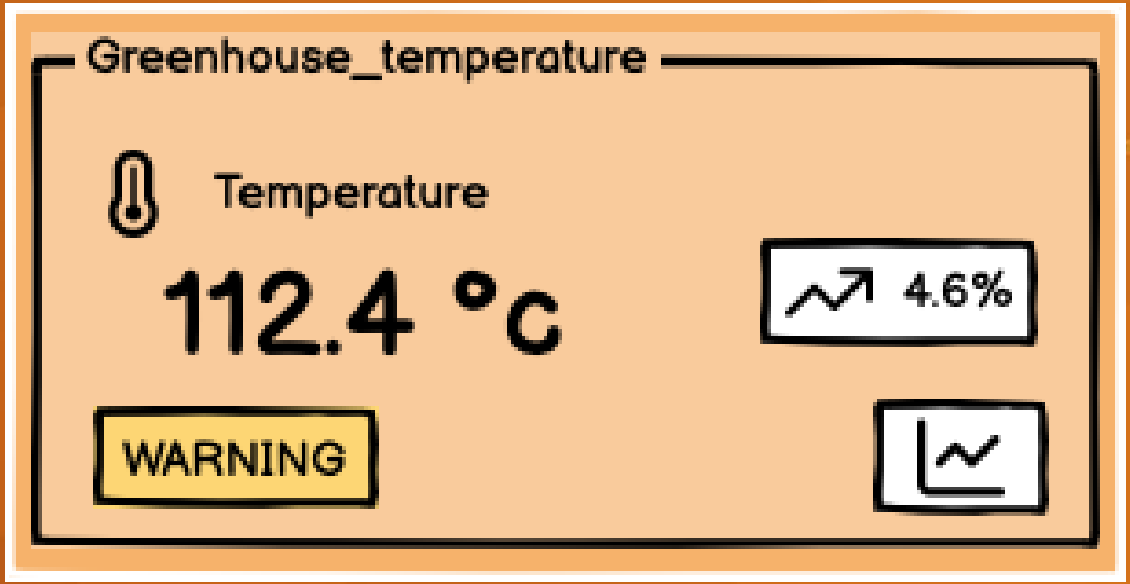
As a user, I want to be able to remove all rules at once.





Suspend all rules

As a user, I want to be able to enable/disable all rules at once.

Sensor status

As a user, I want to be able to see the status of each sensor.